

I need to Implement RK4 for the Nbody problem. What should the state vector be? Is simply running a for loop with RK4 inside on all the particles where the state vector is individual to the particle for each RK4 iteration? OR is it better to run RK4 once and the state vector contains the state of all particles at once?

For the **N-body gravitational problem**, you should run **RK4 once per timestep with a single global state vector that contains all particles at once.**

Not one RK4 per particle.

Why?

The gravitational N-body system is a **coupled system of ODEs**:

$$\dot{\mathbf{v}}_i = \sum_{j \neq i} Gm_j \frac{\mathbf{r}_j - \mathbf{r}_i}{\|\mathbf{r}_j - \mathbf{r}_i\|^3}$$

Each particle's acceleration depends on **every other particle's position** at the same instant in time.

If you:

- Run RK4 independently per particle
- Update particle 1 completely before particle 2

then particle 2 would be using partially updated states. That breaks RK4's consistency and introduces artificial asymmetry and energy drift.

The system must be advanced **as one coupled vector ODE**:

$$\dot{\mathbf{y}} = f(\mathbf{y})$$

✓ Correct Approach

State Vector Structure

For N particles in 2D:

Each particle has:

- Position: x_i, y_i
- Velocity: $v_{x,i}, v_{y,i}$

So total dimension is:

$$4N$$

A clean layout is:

$$\mathbf{y} = \begin{bmatrix} x_1 \\ y_1 \\ x_2 \\ y_2 \\ \vdots \\ v_{x,1} \\ v_{y,1} \\ v_{x,2} \\ v_{y,2} \\ \vdots \end{bmatrix}$$

or equivalently interleaved per particle:

$$[x_i, y_i, v_{x,i}, v_{y,i}] \text{ for } i = 1 \dots N$$

Both are fine — consistency matters more than layout.

The ODE Form

Your system must be first order.

For each particle:

$$\dot{x}_i = v_{x,i}$$

$$\dot{y}_i = v_{y,i}$$

$$\dot{v}_{x,i} = a_{x,i}$$

$$\dot{v}_{y,i} = a_{y,i}$$

So your function:

$$f(\mathbf{y}) \rightarrow \dot{\mathbf{y}}$$

must:

1. Extract all positions
2. Compute all pairwise accelerations
3. Fill the derivative vector

Implementation Pattern

 Code



```
Vector y;           // size 4N
Vector dydt;        // size 4N

function f(y):
    extract positions
    compute all accelerations
    fill dydt
    return dydt

for each timestep:
    y = RK4_step(y, dt, f)
```

Inside RK4:

$$k_1 = f(y)$$

$$k_2 = f\left(y + \frac{dt}{2} k_1\right)$$

$$k_3 = f\left(y + \frac{dt}{2} k_2\right)$$

$$k_4 = f(y + dt k_3)$$

$$y_{n+1} = y_n + \frac{dt}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

All particles are advanced simultaneously.

✗ Why Per-Particle RK4 is Wrong

If you do:

Code

```
for each particle i:
    RK4_step(particle_i)
```



Then:

- Particle 2 sees particle 1 already partially updated.
- Time symmetry breaks.
- Energy drift increases.
- Results become timestep-order dependent.

This is mathematically inconsistent for coupled systems.

🎯 The Clean Mental Model

The N-body system is:

$$\dot{\mathbf{y}} = f(\mathbf{y})$$

where

$$\mathbf{y} \in \mathbb{R}^{4N}$$

It is **one big ODE**, not N small ones.

🚀 Extra Insight (Important for Physics Simulations)

RK4 works, but for gravitational systems it does **not conserve energy** long term.

For orbital mechanics, symplectic integrators like:

- Leapfrog (Velocity Verlet)
- Stormer–Verlet

are usually better.

RK4 is higher local accuracy, but worse long-term structure preservation.

Since you're interested in orbital mechanics and simulation work, this is something you'll eventually want to care about.



Final Answer

Use:

One RK4 step per timestep with a state vector containing all particles.

Not:

RK4 inside a loop over particles.

If you'd like, I can next:

- Show the clean mathematical structure for the full $4N$ system
- Or show how to organize this cleanly in Eigen without turning your code into spaghetti

Which direction do you want to go?



I need to set up an enum class to switch between the different numerical integrators I want to use. I want to use Euler step, Leapfrog, and RK4

Good — this is exactly the right time to design this cleanly.

You want:

- A type-safe way to select the integrator
- A clean switch point
- No duplicated simulation logic
- Easy extensibility (so you can add e.g. Verlet, RK2, adaptive RK, etc.)

✓ Step 1: Define the `enum class`

</> C++



```
enum class Integrator
{
    Euler,
    Leapfrog,
    RK4
};
```

Why `enum class` ?

- Scoped (no global name pollution)
- Strongly typed
- Prevents accidental implicit conversions

✓ Step 2: Structure Your Simulation Cleanly

You want:

- One global state vector `y`
- One acceleration function
- Separate step functions per integrator

A clean pattern:

</> C++



```
void stepEuler(State& state, double dt);
void stepLeapfrog(State& state, double dt);
void stepRK4(State& state, double dt);
```

Then:

```
</> C++  
  
void integrate(State& state, double dt, Integrator method)  
{  
    switch (method)  
    {  
        case Integrator::Euler:  
            stepEuler(state, dt);  
            break;  
  
        case Integrator::Leapfrog:  
            stepLeapfrog(state, dt);  
            break;  
  
        case Integrator::RK4:  
            stepRK4(state, dt);  
            break;  
    }  
}
```

This keeps the integrator selection logic centralized.

🔥 Even Better: Function Pointer Dispatch (Cleaner)

Instead of switching every frame, bind once:

```
</> C++  
  
using IntegratorFunc = void (*)(State&, double);  
  
IntegratorFunc selectIntegrator(Integrator method)  
{  
    switch (method)  
    {  
        case Integrator::Euler:    return stepEuler;  
        case Integrator::Leapfrog: return stepLeapfrog;  
        case Integrator::RK4:      return stepRK4;  
    }  
  
    return stepRK4; // fallback  
}
```

Then in your main loop:

```
</> C++  
  
auto integrator = selectIntegrator(method);  
  
while (running)  
{  
    integrator(state, dt);  
}
```

Now there's:

- No switch inside the loop
- Clean extensibility
- Nice separation of concerns

Important Architectural Note

Since you're doing N-body:

Euler and RK4 operate on full state vectors

Leapfrog operates naturally on split position/velocity form

So your `State` structure matters.

You likely want something like:

```
</> C++  
  
struct State  
{  
    Eigen::VectorXd positions;    // size 2N  
    Eigen::VectorXd velocities;  // size 2N  
    Eigen::VectorXd masses;      // size N  
};
```

Why?

Because Leapfrog requires:

$$v_{n+1/2} = v_n + \frac{dt}{2} a(x_n)$$

which is awkward if everything is flattened into one 4N vector.



Clean Design Pattern for You

Since you're building a physics engine style sim, this is the scalable way:

1. State holds data only
2. Acceleration function computes forces
3. Integrators modify state
4. Main loop calls chosen integrator

This is how small physics engines are structured.



Final Recommendation

Use:

⟨⟩ C++



```
enum class Integrator
{
    Euler,
    Leapfrog,
    RK4
};
```

Use a `selectIntegrator()` function.

Store positions and velocities separately (better for Leapfrog).

If you want, next I can:

- Show clean minimal Euler, Leapfrog, and RK4 implementations side-by-side for N-body

- Or help you design this in a way that scales to 3D and collision later

Given your interest in orbital mechanics, I'd honestly recommend structuring this like a tiny physics engine now rather than patching it later.



for Eigen I want to loop through the rows of matrix, setting the row for each iteration of the loop

In **Eigen**, looping through rows and setting them is straightforward using `.row(i)`.

Assume:

<> C++



```
Eigen::MatrixXd M(rows, cols);
```

✓ Setting Rows in a Loop

◆ Method 1: Assign a Vector to Each Row

<> C++



```
for (int i = 0; i < M.rows(); ++i)
{
    Eigen::RowVectorXd r = Eigen::RowVectorXd::Constant(M.cols(), i);
    M.row(i) = r;
}
```

Key point:

- `M.row(i)` returns a **RowVector** expression
- The sizes must match

◆ Method 2: Set Individual Elements

</> C++



```
for (int i = 0; i < M.rows(); ++i)
{
    for (int j = 0; j < M.cols(); ++j)
    {
        M(i, j) = someFunction(i, j);
    }
}
```

◆ Method 3: Using `setConstant`, `setZero`, etc.

</> C++



```
for (int i = 0; i < M.rows(); ++i)
{
    M.row(i).setConstant(i);
}
```

or

</> C++



```
M.row(i).setZero();
```

Important Eigen Gotcha

`row(i)` returns an expression, not a copy.

So this is WRONG:

</> C++



```
Eigen::RowVectorXd r = M.row(i); // This makes a copy
r(0) = 5;                        // Does NOT modify M
```

If you want a reference:

</> C++



```
auto r = M.row(i); // this keeps expression reference
r(0) = 5;          // modifies M
```

or explicitly:

</> C++



```
Eigen::RowVectorXd::MapType r = M.row(i); // rarely needed
```

Usually `auto` is fine.



If You're Doing This for N-Body

If your matrix is:

- positions as $N \times 2$
- velocities as $N \times 2$

Then this is very clean:

</> C++



```
for (int i = 0; i < N; ++i)
{
    positions.row(i) += dt * velocities.row(i);
}
```

That's much cleaner than flattening into $4N$ vectors for simple integrators like Euler.

If you'd like, I can also show:

- The fastest Eigen-friendly way to compute pairwise forces
- Or how to vectorize the N-body force computation

Since you're building a simulator, small structural decisions now matter a lot later.



```
Eigen::Vector<float, NUM_PARTICLES> K1;  
I am having trouble with this line because NUM_PARTICLES  
= particles.size() which is a std::vector
```

Ah — this is a **compile-time vs runtime size** issue in Eigen.

Why This Fails

 C++



```
Eigen::Vector<float, NUM_PARTICLES> K1;
```

Eigen expects the template size to be:

- A compile-time constant
- Known at compile time

But:

 C++



```
NUM_PARTICLES = particles.size();
```

is:

- Runtime value
- Determined after the program starts
- Not `constexpr`

So the compiler cannot instantiate that type.

Correct Solution (Dynamic Size)

Use `Eigen::VectorXf` :

 C++

```
Eigen::VectorXf K1(particles.size());
```

or equivalently:

 C++

```
Eigen::Vector<float, Eigen::Dynamic> K1(particles.size());
```

That is how you create a runtime-sized vector.

Why This Matters for Your N-Body

Since:

- Number of particles is not fixed at compile time
- You may add/remove particles
- You may change N

You should almost certainly be using **dynamic Eigen vectors/matrices**.

For example:

 C++

```
Eigen::MatrixXf positions(N, 2);  
Eigen::MatrixXf velocities(N, 2);
```

or flattened:

 C++

```
Eigen::VectorXf state(4 * N);
```

When Would Fixed Size Work?

This would work:

</> C++



```
constexpr int NUM_PARTICLES = 100;  
Eigen::Vector<float, NUM_PARTICLES> K1;
```

But only if:

- The size is truly fixed
- Known at compile time
- Never changes

That's common in embedded systems or small physics demos, but usually not for simulations like yours.

Best Practice for You

Since you're building an N-body simulator:

Use:

</> C++



```
const int N = particles.size();  
Eigen::VectorXf K1(4 * N);  
Eigen::VectorXf K2(4 * N);  
Eigen::VectorXf K3(4 * N);  
Eigen::VectorXf K4(4 * N);
```

Or, even better — allocate once and reuse them each timestep to avoid heap churn.

If you'd like, I can show:

- A clean RK4 layout using `VectorXf`
- Or how to structure this so you don't reallocate every frame (important for performance)

You're asking exactly the right questions for writing a clean simulator.



how do I improve these lines of code to potential reduce it to one line:

```
K1[2*idx] = gravity.p[idx].velocity[0];  
K1[2*idx+1] = gravity.p[idx].velocity[1];
```

idx is just a for loop iterator

Good — this is exactly where Eigen expressions shine.

Right now you're manually copying components:

</> C++



```
K1[2*idx] = gravity.p[idx].velocity[0];  
K1[2*idx+1] = gravity.p[idx].velocity[1];
```

That's scalar-style thinking. Since velocity is a 2D vector, you should move both components at once.

✓ Best Eigen Way (Block Assignment)

If `k1` is a `VectorXf` :

</> C++



```
K1.segment<2>(2 * idx) = gravity.p[idx].velocity;
```

That replaces both lines with one.

What This Does

`segment<2>(2 * idx)` means:

- Take 2 consecutive elements

- Starting at index `2*idx`

So it writes into:

`</>` Code



```
[2*idx, 2*idx + 1]
```

Exactly what you were doing manually.

If Size Is Runtime (Still Works)

If for some reason the segment length isn't known at compile time:

`</>` C++



```
K1.segment(2 * idx, 2) = gravity.p[idx].velocity;
```

But since it's always 2, prefer the compile-time version `<2>` — it's slightly more efficient.

Even Cleaner (If You Control Layout)

If your state vector stores positions and velocities like:

`</>` Code



```
[x_i, y_i, vx_i, vy_i]
```

Then you can do:

`</>` C++



```
K1.segment<2>(4 * idx + 2) = gravity.p[idx].velocity;
```

Which keeps your layout consistent and clean.

Why This Is Better

- Fewer lines
 - No index arithmetic duplication
 - Less error-prone
 - Vectorized by Eigen
 - Easier to refactor later
-

If you'd like, I can show you how to eliminate even the `2*idx` arithmetic entirely using a `Map` or by restructuring your state storage.

